

TRABAJO PRÁCTICO 2

Curvas de Bézier y Optimización

Métodos Computacionales

Autores: Iñaki Irarrazaval - Manuel Ferrario

Contenidos

I

Representación de curvas de Bézier

- Ej 1 — Funciones base de Bernstein
- Ej 2 — Evaluación matricial
- Ej 3 — Cálculo de la matriz C

II

Mínimos cuadrados

- Ej 4 — Ajuste por mín. cuadrados
- Ej 5 — Error como forma cuadrática
- Ej 6 — Ecuaciones normales

III

Métodos de optimización

- Ej 7 — Gradiente descendiente
- Ej 8 — Tasa de aprendizaje
- Ej 9 — Método de Newton
- Ej 10 — Extremos fijos

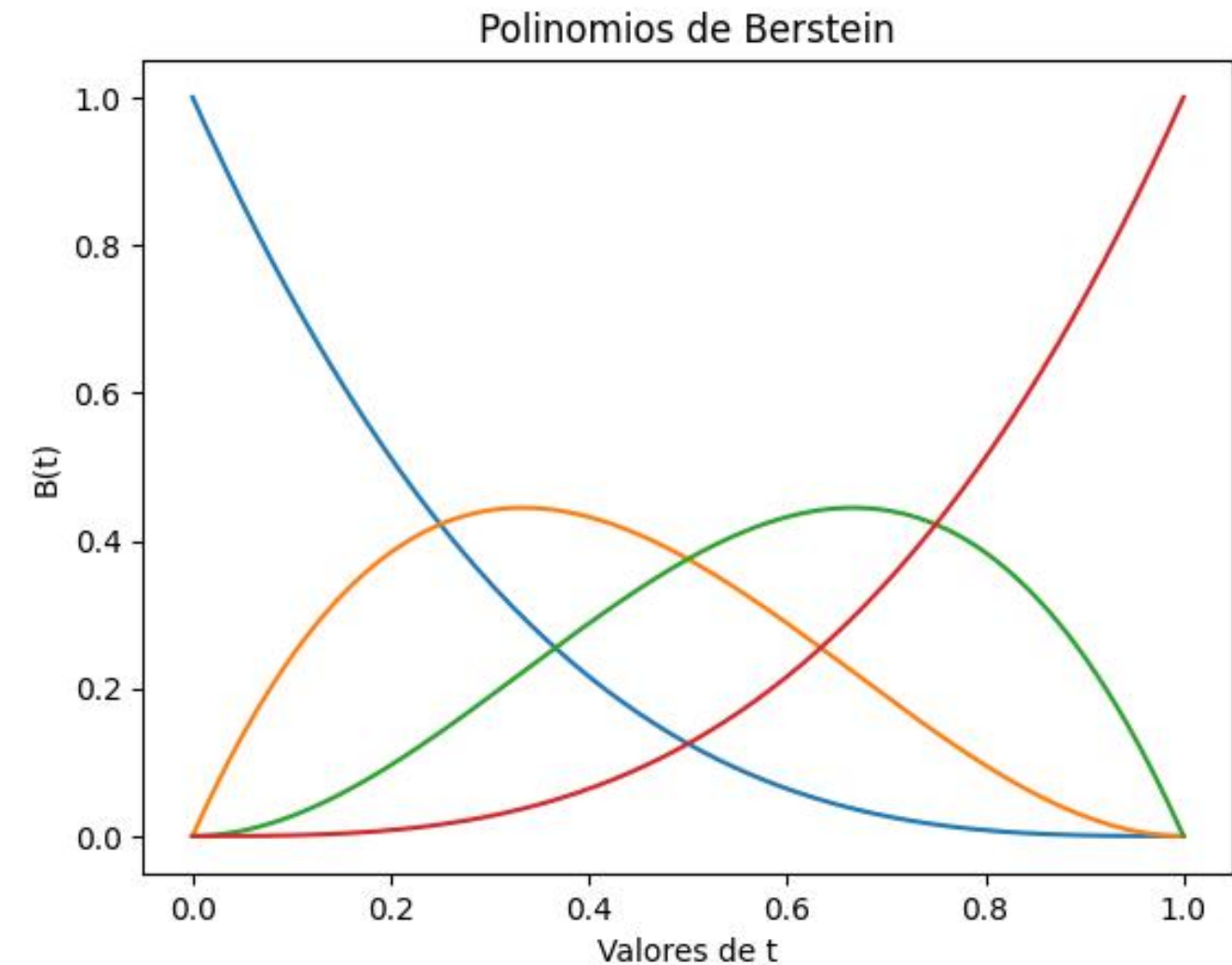
IV

Aplicación: podadora robot

- Ej 11 — Trayectoria para el robot
- Ej 12 — Optimización del grado
- Ej 13 — Análisis de la trayectoria

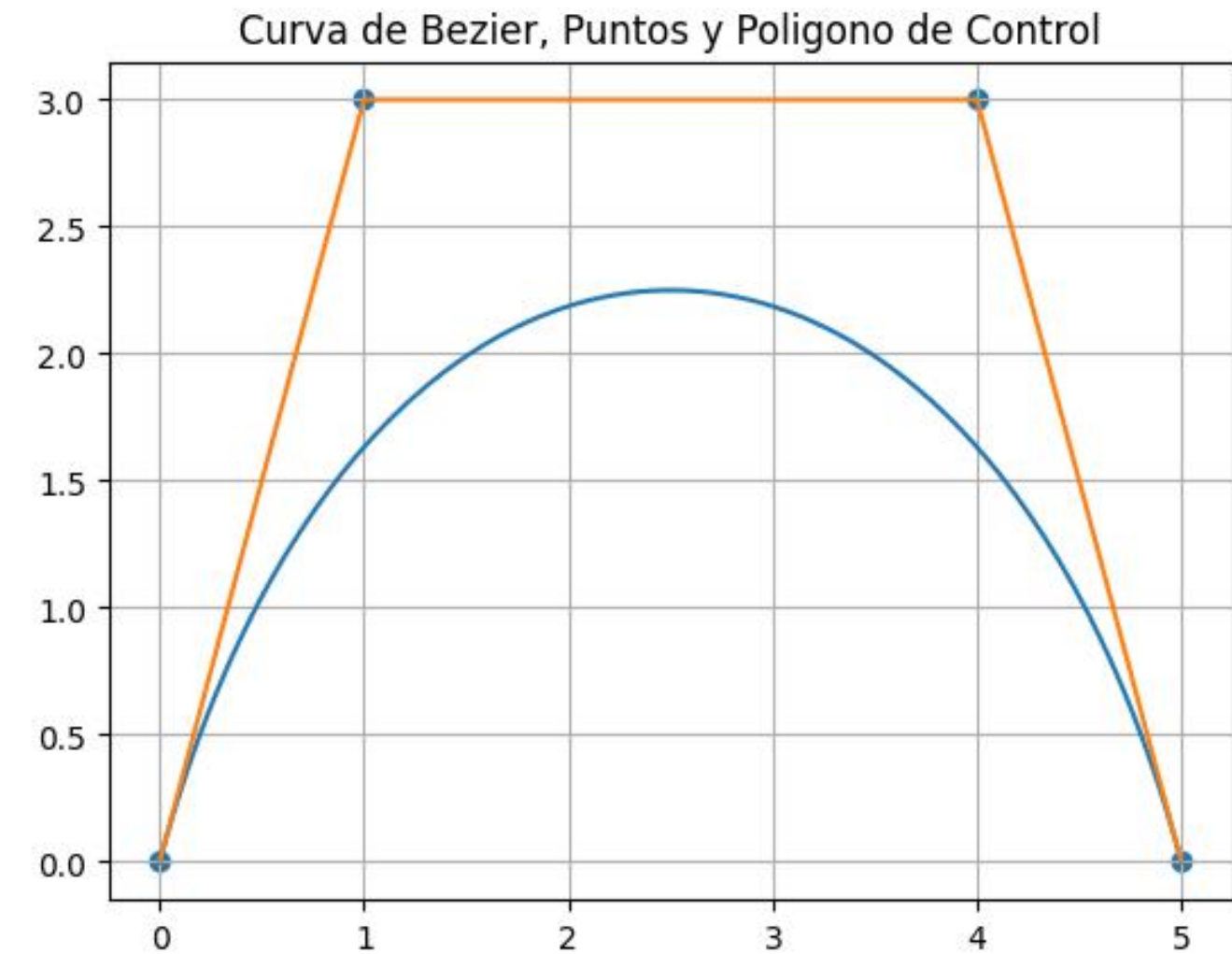
Funciones base de Bernstein (grado 3)

- Generamos una matriz A que nos acompañara durante todo el trabajo practico.
- La matriz A es una matriz con filas|valores_t| y columnas tamaño 4.
- Cada valor de la matriz el polinomio de Berstein remplazando t.
- Los polinomios de Berstein son una combinación de puntos de control.
- Esto hace que la curva de Bezier mantenga su forma sin importar del origen de las coordenadas.
- La curva siempre estara dentro del poligono de control.



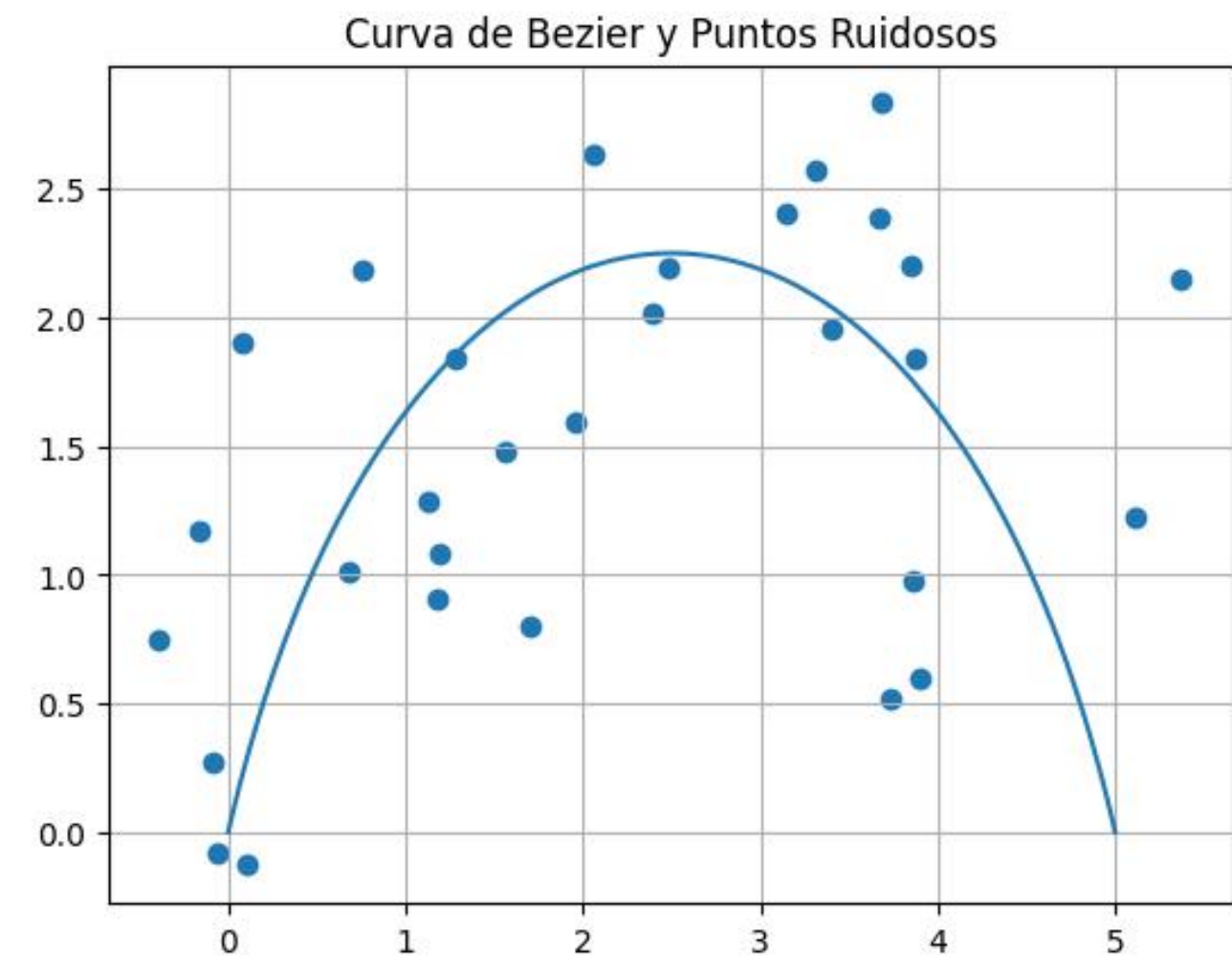
Evaluación matricial de una curva de Bézier

- Construimos una matriz P con los puntos de control, de tamaño 4×2 .
- Calculamos $Q = A * P$.
- Graficamos la curva Q , los puntos de control de P y el polígono de control.



Generación de datos con ruido

- Evaluamos la curva original con filas de A tamaño 30.
- Generamos vectores que representarían ruido gaussiano.
- Le sumamos a Q los puntos de ruido.
- Graficamos la curva original con los puntos con ruido.

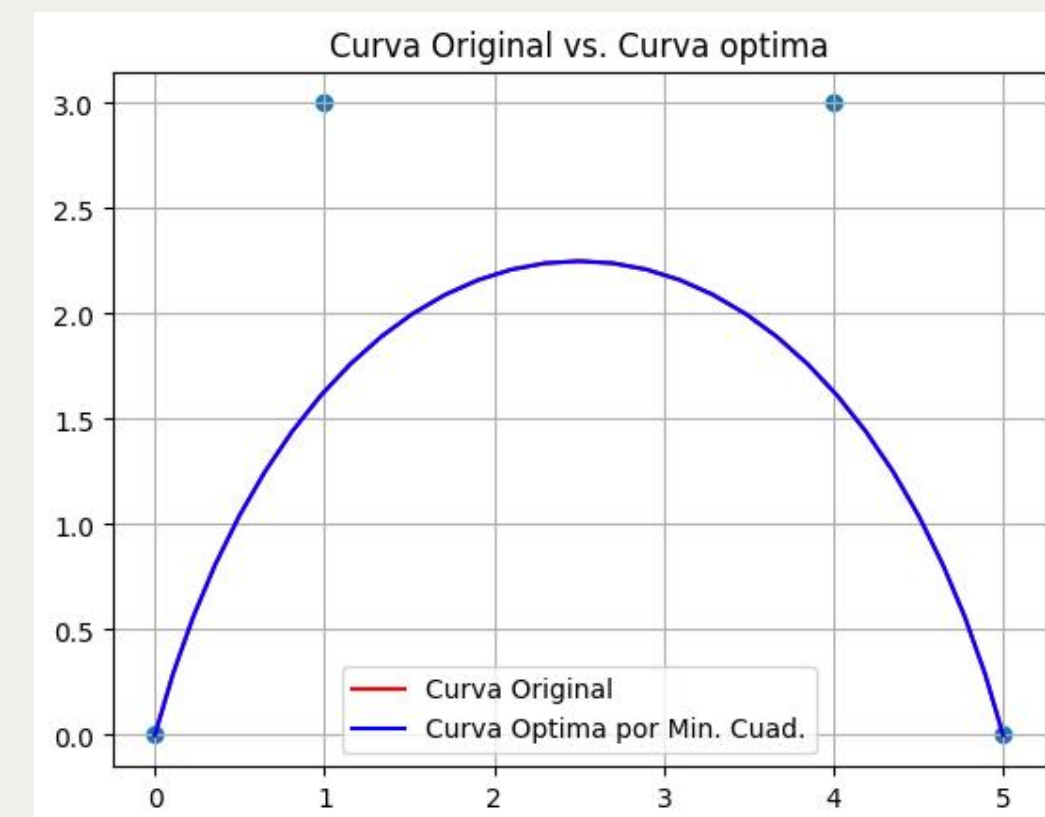
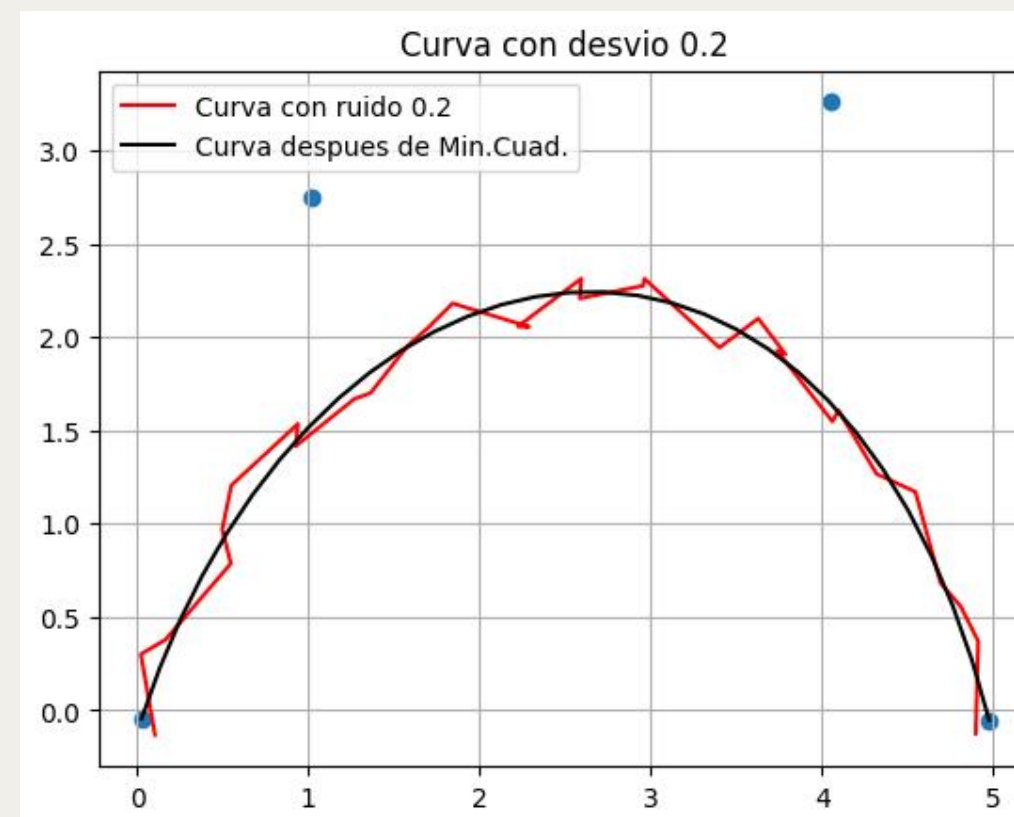
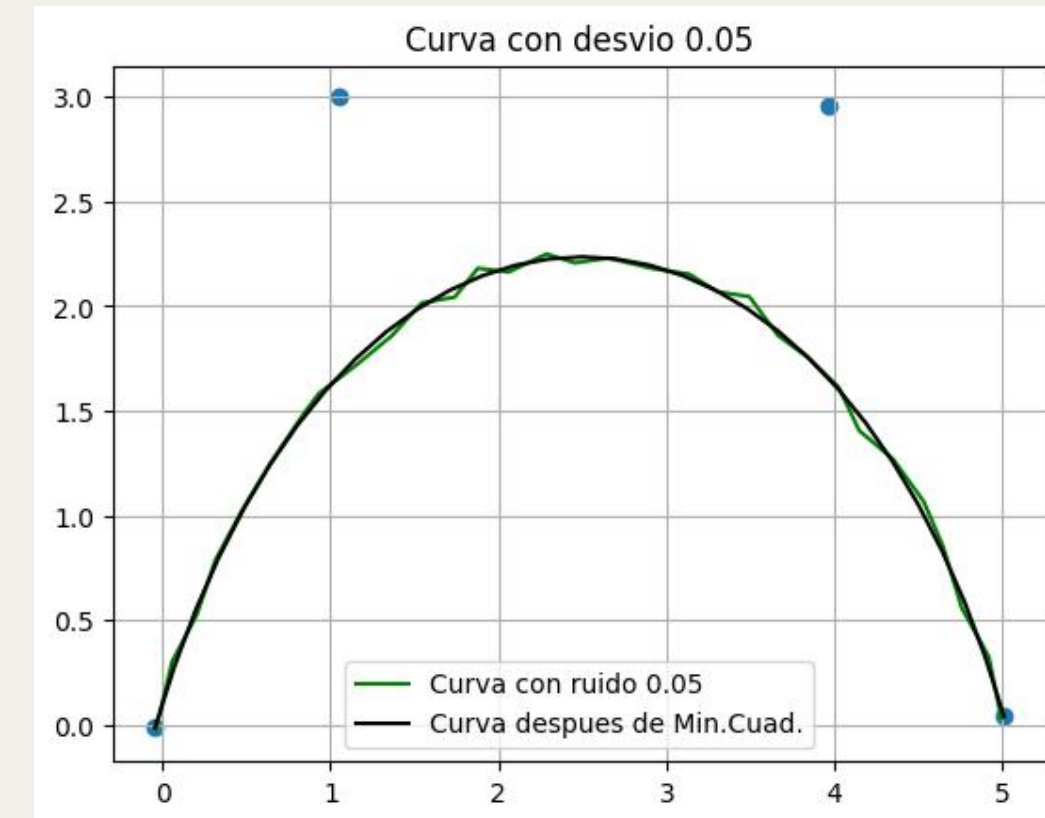


PARTE II — EJERCICIOS 4 A 6

Mínimos cuadrados

Ajuste por mínimos cuadrados

- Tenemos dos curvas con ruido
 - Una con desvio de ruido de 0.2 y 0.05.
- Vemos que al ajustar las curvas con ruido por minimos cuadrados, obtenemos una aprox. a los puntos de control originales.
- Al aplicar ruido, nos movemos a otro subespacio y con Ecu. Norm. encontramos la proyeccion ortogonal.
- Calculamos ECM y vemos que para desvios mas grandes sube el error. Cuando hacemos min. cuad. el error es casi cero.



El error como forma cuadrática

(a)

$$\|A p - q\|^2 = \sqrt{(A p - q)^T (A p - q)}^2 = (A p - q)^T \cdot (A p - q) = (p^T A^T - q^T) \cdot (A p - q) = p^T A^T A p - p^T A^T q - A p q + q^T q = p^T A^T A p - 2 q^T A p + q^T q$$

(b) La parte cuadrática de la expresión es la $p^T A^T A p$ ya que sigue la forma de $Q(x) = x^T A x$ siendo $A^T A$ la matriz simétrica.

(c) ¿Porque $A^T A$ es simétrica?

Demo: si $A^T A$ es simétrica $A^T A = (A^T A)^T$:

$$\Rightarrow A^T A = (A^T A)^T = A^T \cdot (A^T)^T = A^T A.$$

$\therefore$ Es simétrica.

- Calculamos los valores propios de $(A^t * A)$ y son todos positivos. Como son positivos, es una función positiva definida.
- Como es positiva definida va a tener mínimo global.
- Como tiene mínimo global, lo vamos a encontrar con Metodo Newton o Gradiente Descendiente.

Ecuaciones normales desde el gradiente

- Calculamos el gradiente del error de minimos cuadrados.
- El gradiente indica hacia donde crece la funcion cuadratica.
- Como vamos a buscar el minimo, nos movemos al revez que el gradiente.
- Cuando ya tenemos la solución de minimos cuadrados, el gradiente va a ser casi cero.

$$\begin{aligned}
 & e) \text{ si } \nabla \mathcal{E}(p) = 2A^T(Ap - q) \\
 & \text{si igualamos } \nabla \mathcal{E}(p) = 0 \Rightarrow 0 = 2A^T(Ap - q) \xLeftrightarrow{\text{distributiva}} 0 = 2A^T \cdot Ap - 2A^T \cdot q \xLeftrightarrow{\times 2} 2A^T \cdot Ap = 2A^T q \Leftrightarrow A^T \cdot A \cdot p = A^T \cdot q \\
 & \hspace{25em} \hookrightarrow \text{ecuacion de minimos cuadrados.}
 \end{aligned}$$

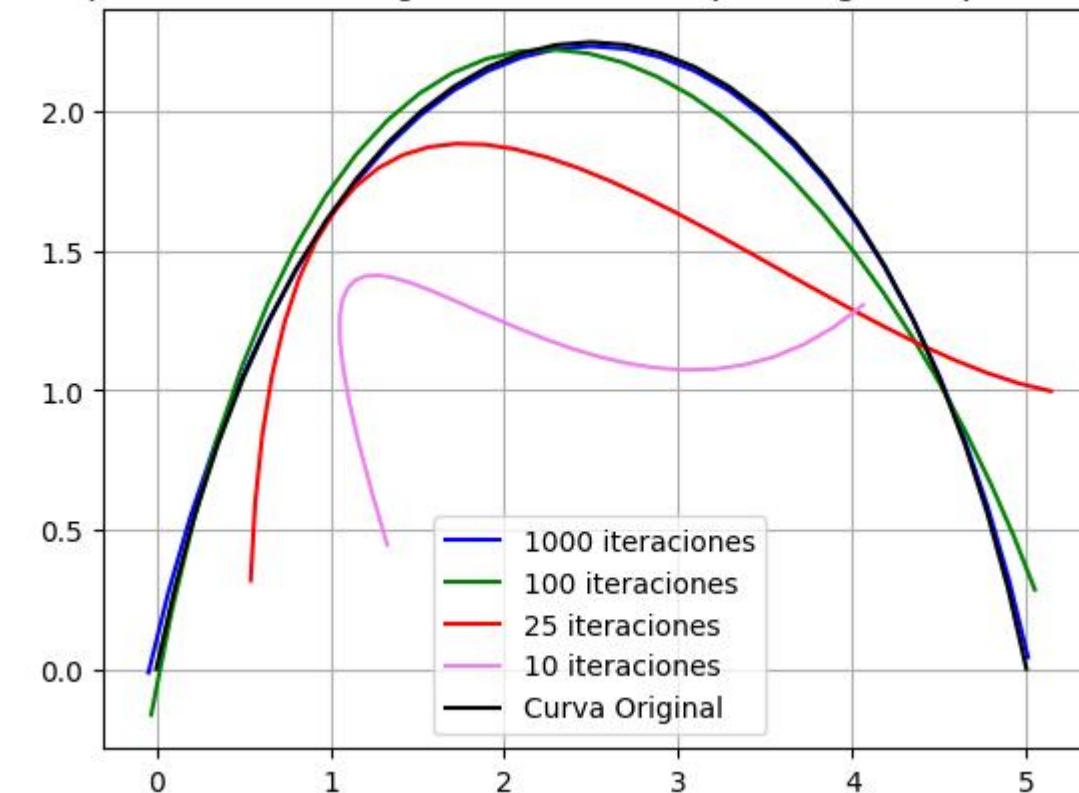
Gradiente descendiente

- Gradiente descendiente es un algoritmo para encontrar el mínimo o máximo de una función. Mediante pequeñas iteraciones para ver hacia donde va el min/max.

$$p_{t+1} = p_t - \alpha 2A^T(Ap_t - q).$$

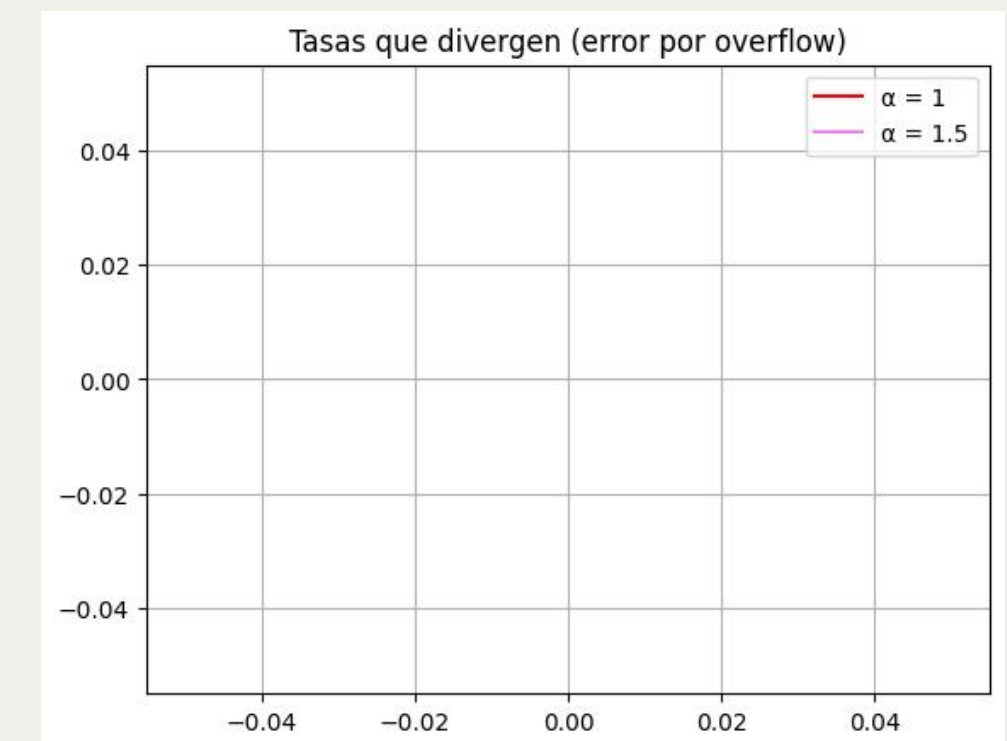
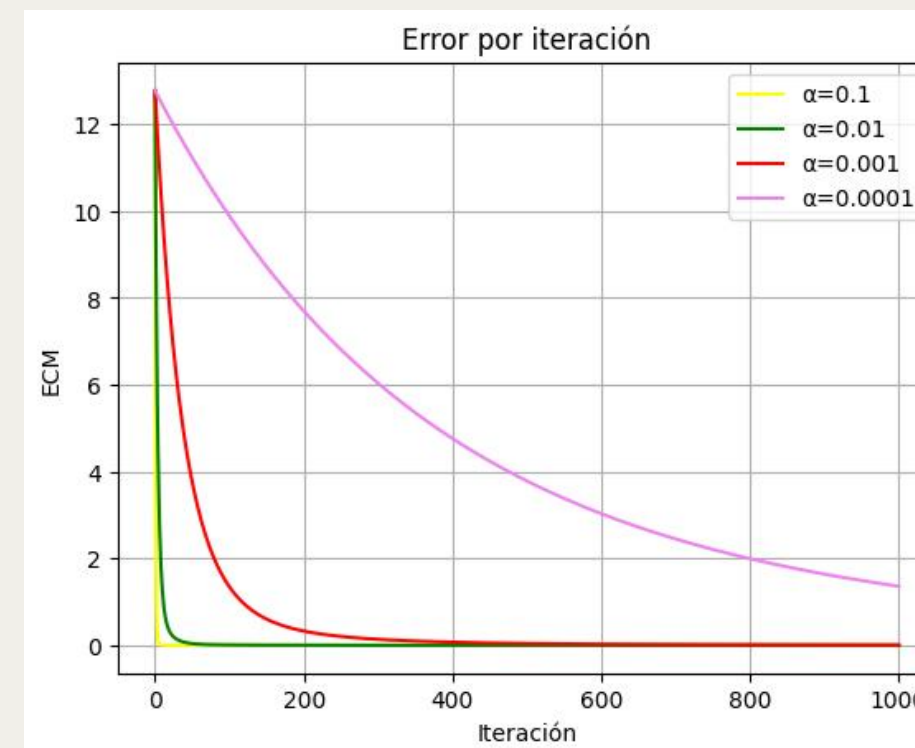
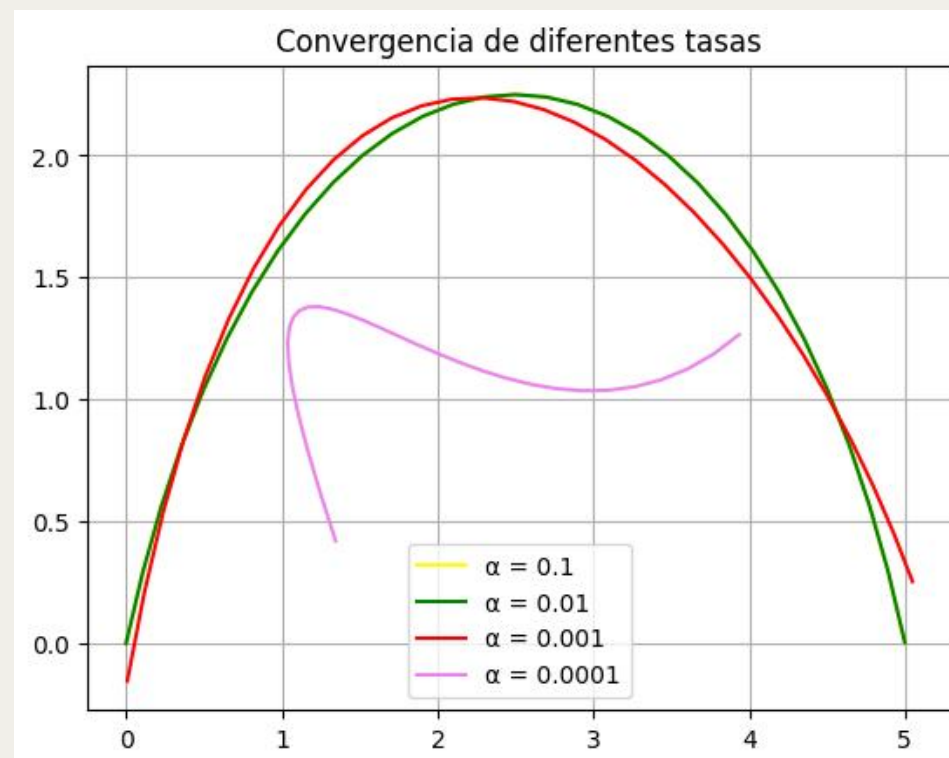
- Esta actualización hace movernos hacia donde no va el gradiente. Entonces buscamos el mínimo del error.
- α es nuestra tasa de aprendizaje (usamos 0.01 para las iteraciones del gráfico).

Desde un punto random --> ¿Cuanto tardamos para llegar al optimo? Grad. Desc.



Efecto de la tasa de aprendizaje

- Probar 4 valores de tasa de aprendizaje (gráfico 1).
- Vemos como el error va bajando en base a las iteraciones (gráfico 2)
- Para $\alpha = 0.0001$ converge lentamente.
- En casos donde la tasa es muy grande, diverge y nunca encuentra mínimo.
- La tasa de aprendizaje es cuanto “descubrimos” del mínimo en cada iteración.

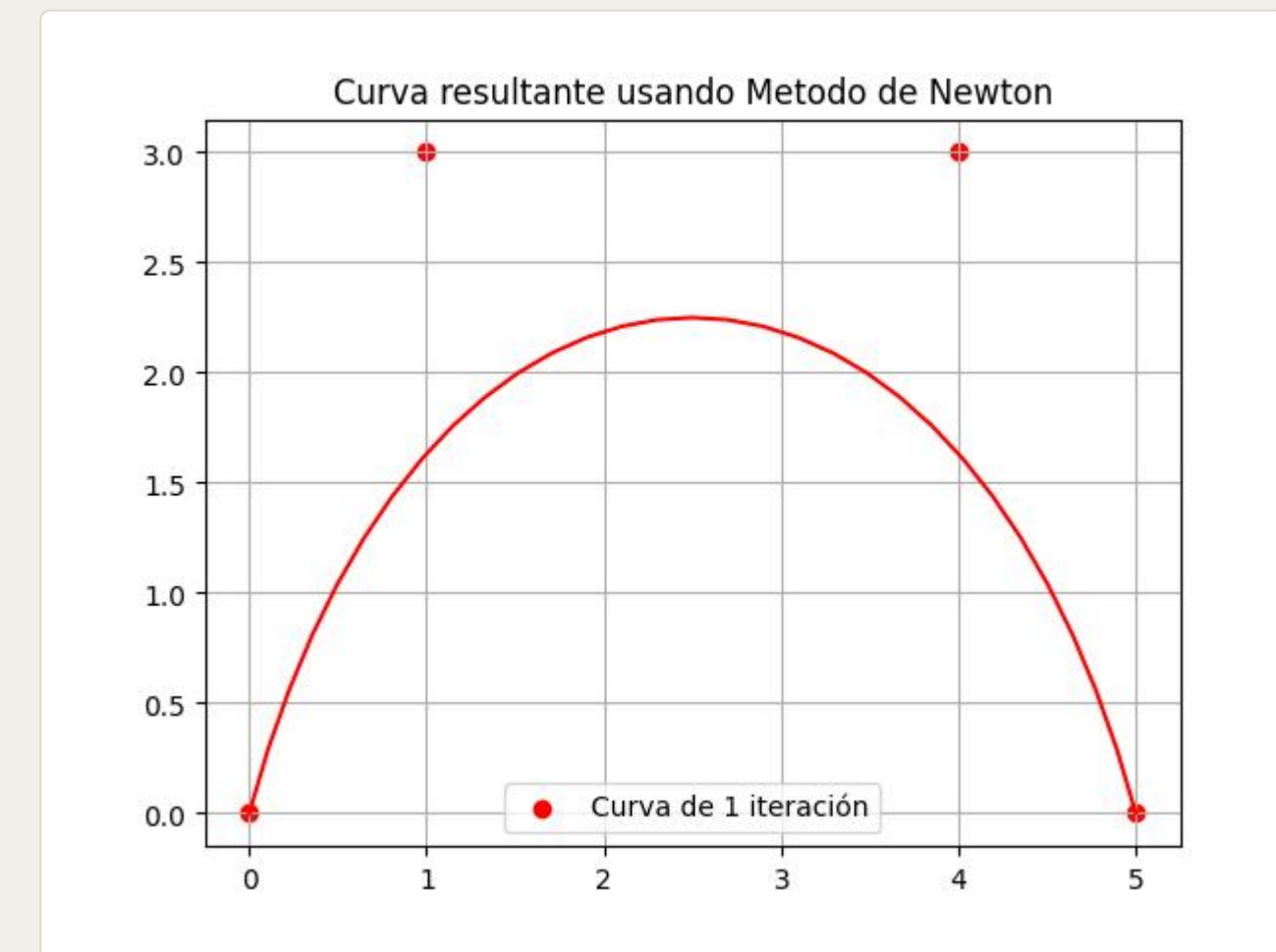


Método de Newton

- El Hessiano es la doble derivada.

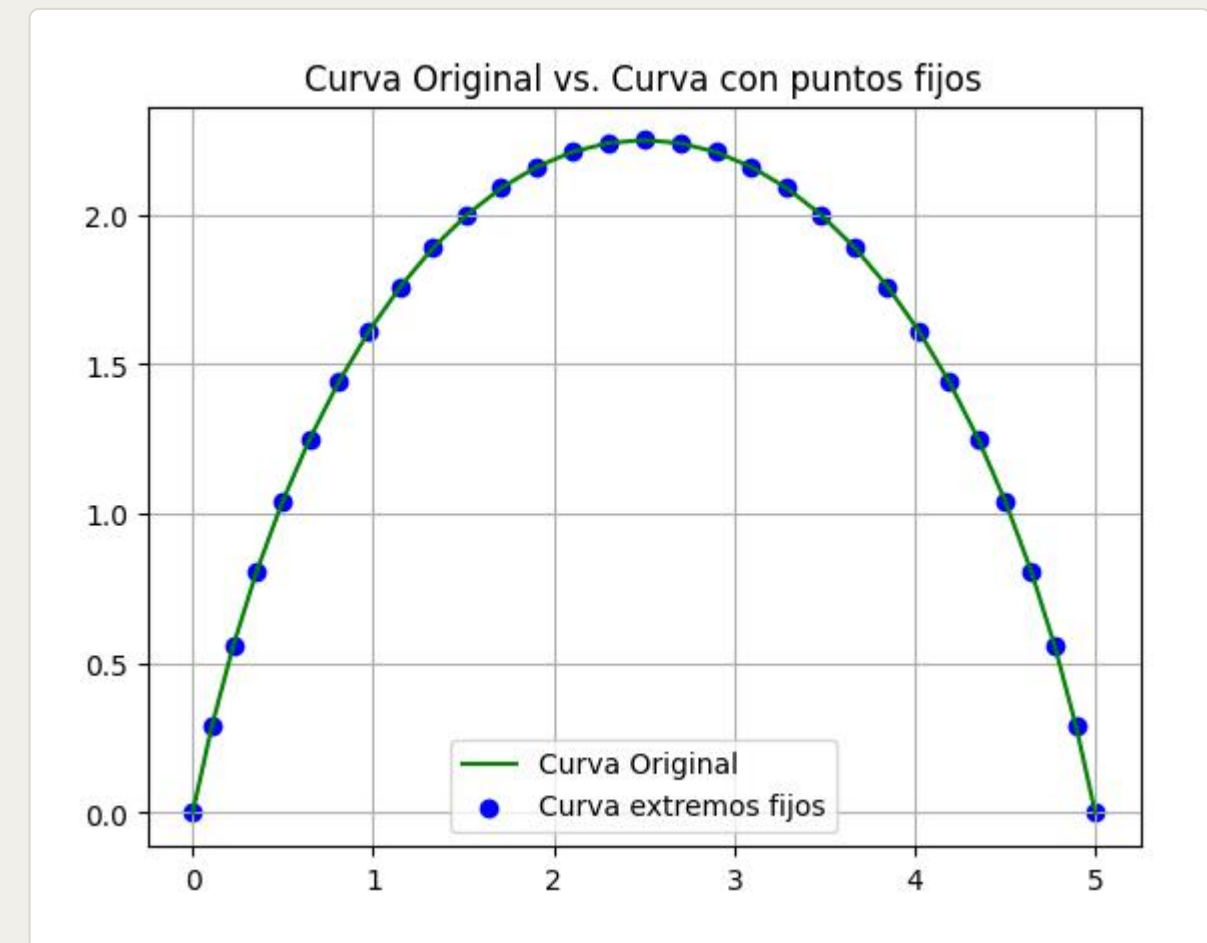
$$p_{t+1} = p_t - H^{-1} \nabla E(p_t).$$

- Esta actualización es la que realizamos en el Metodo de Newton.
- Newton llega en una iteración a la solución optima porque el error es una cuadratica. (Sino tardaria más).
- Digamos que, Newton hace saltos mas justos que Gradiente Descendiente, por la Hessiana.
- Lo que si, Newton cuesta mucho mas computacionalmente que el Gradiente. Ya que, calcula una inversa.



Ajuste con extremos fijos

- Fijar extremos (P0 y P3).
- Tenés una “nueva” curva que solo depende de (P2 y P1).
- Estimando la curva de extremos fijos, vemos que es la misma que la curva original (gráfico).



PARTE IV — EJERCICIOS 11 A 13

Aplicación: podadora robot

Trayectoria para la podadora robot

$$B_n(t) = \sum_{k=0}^n b_{k,n}(t) P_k, \quad b_{k,n}(t) = \binom{n}{k} (1-t)^{n-k} t^k, \quad t \in [0, 1].$$

- Implementamos una función que dado un grado n y valores de t genere una los valores de los polinomios de berstein (A).
- Implementamos una función que recibe P y los valores de t. Nos devuelve Q.

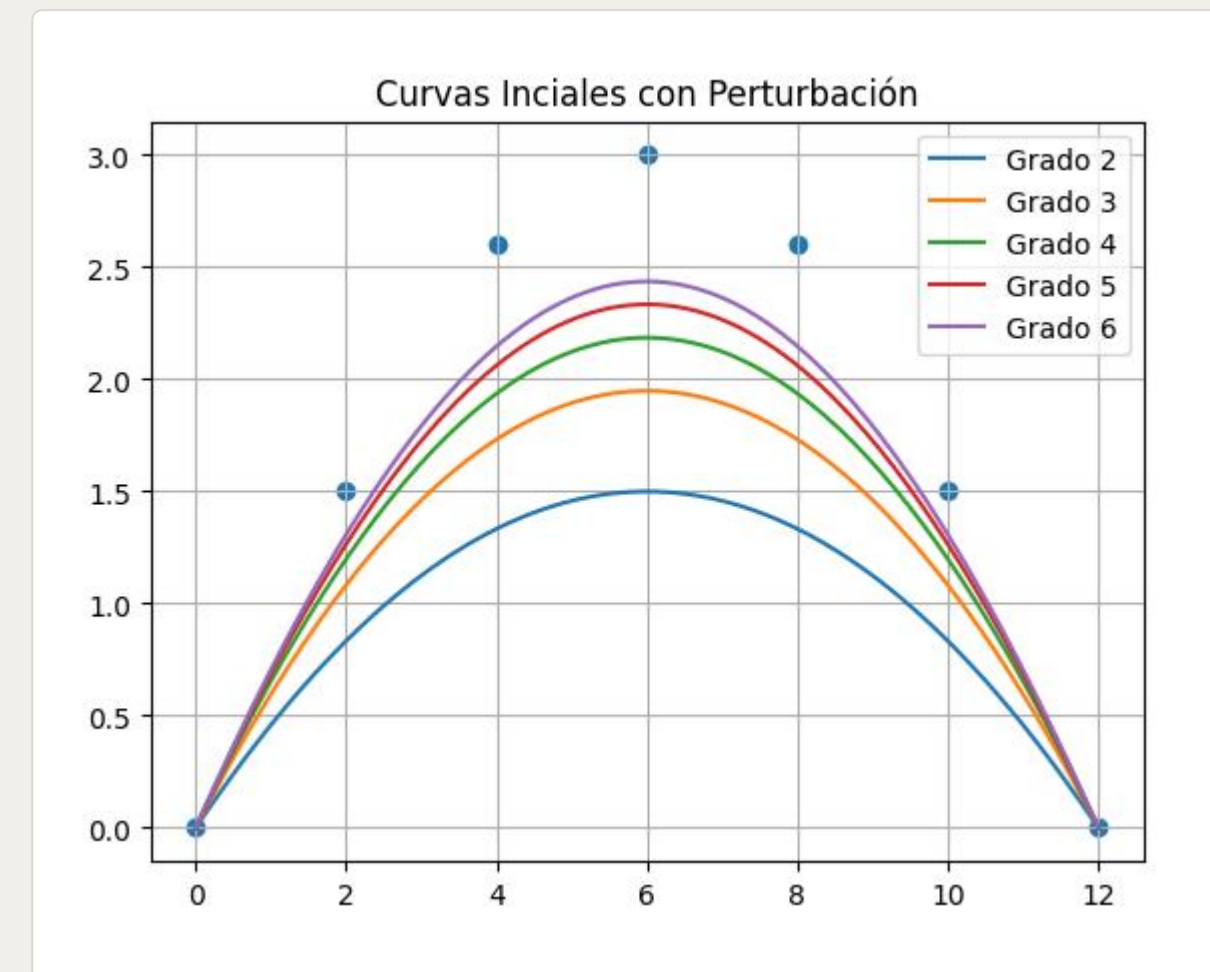
$$\min_i \|B_n(t_i) - c_j\| < r_j.$$

- La función colision() devuelve si hubo una colisión con los objetos.
- Inicializamos curvas con la perturbación. El a de la perturbación determinaba cuanto arriba se va nuestra curva. (a=3)

$$P_k^{(0)} = \left(12 \frac{k}{n}, a \sin \left(\pi \frac{k}{n} \right) \right), \quad k = 0, \dots, n,$$

- El gráfico representa las curvas iniciales con distintos grados.

Todas estas chocaban con algún obstáculo.



Optimización automática

- Fijamos O y D.
- Ahora θ va a ser nuestra vector con puntos.
- Definimos las función de error (ErrorLargo + LandaSuav*ErrorSuav + LandaObs*ErrorObs)

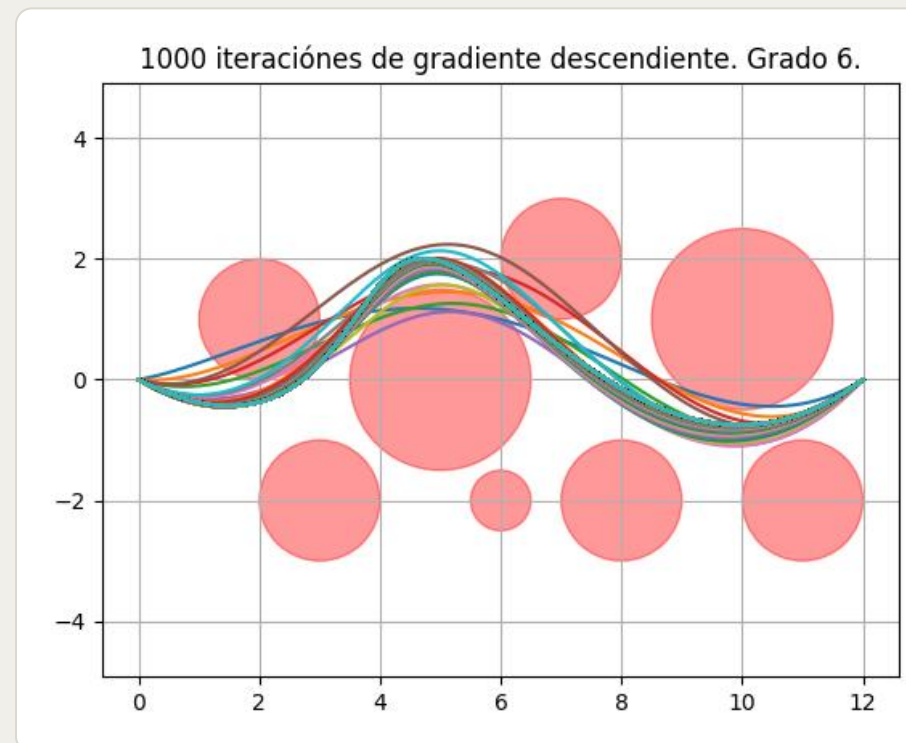
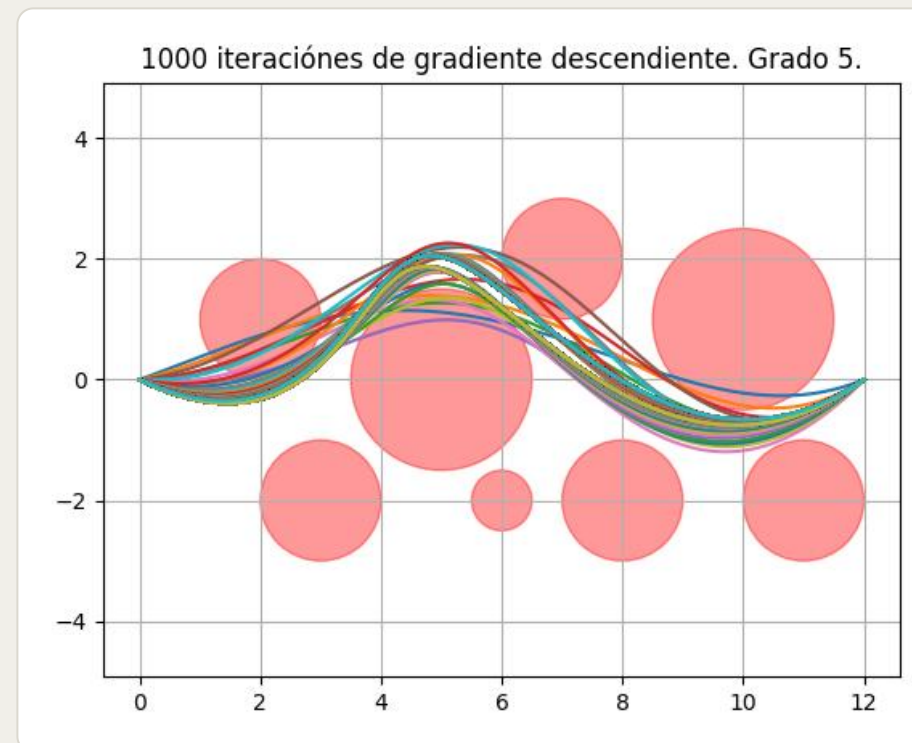
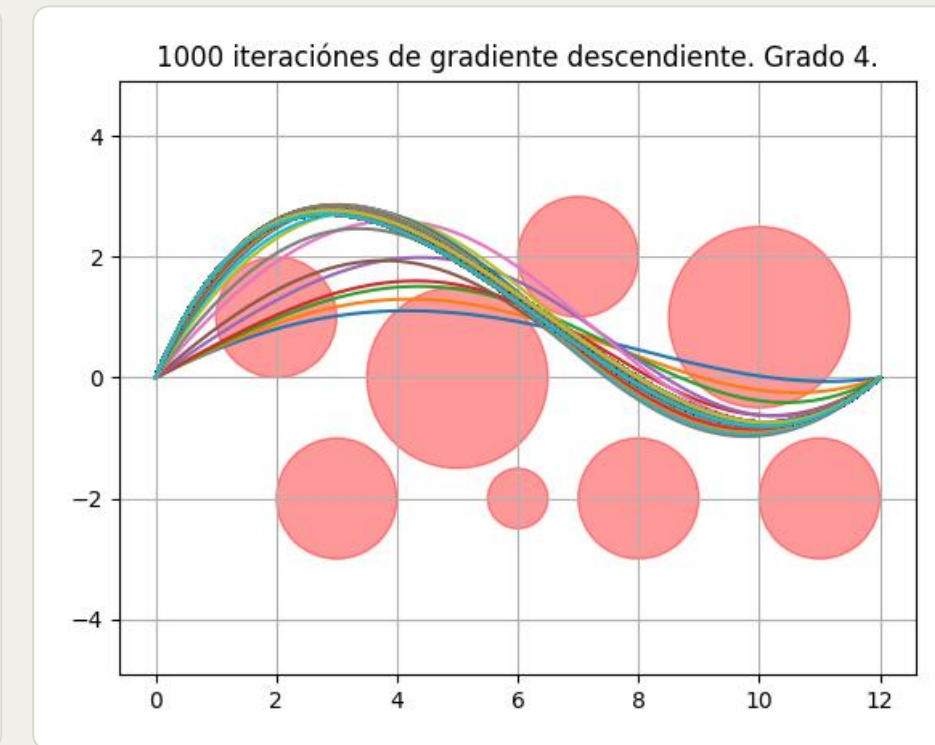
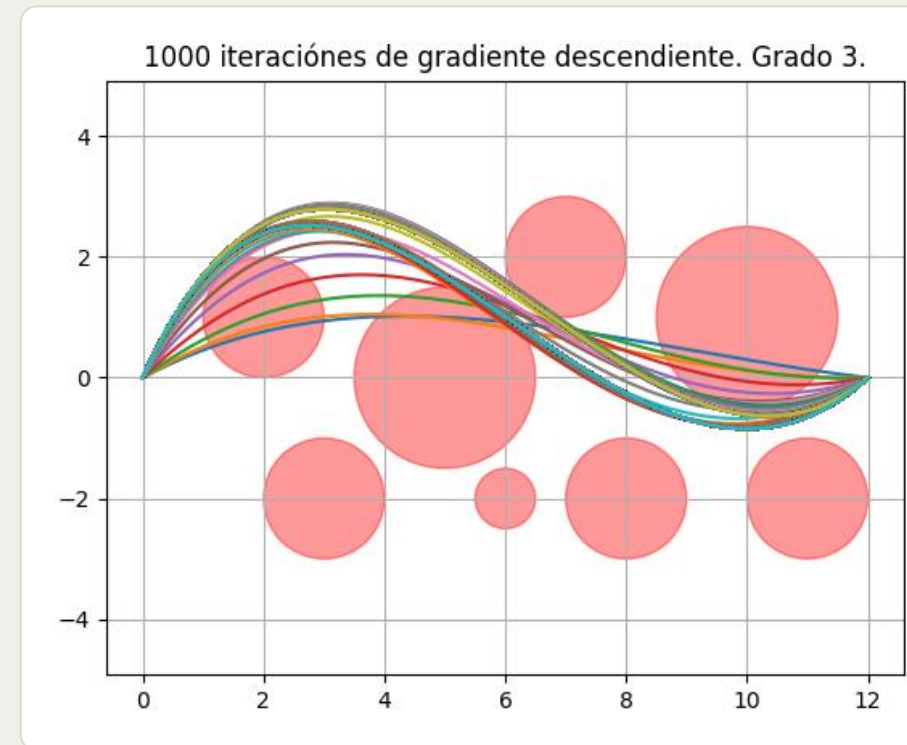
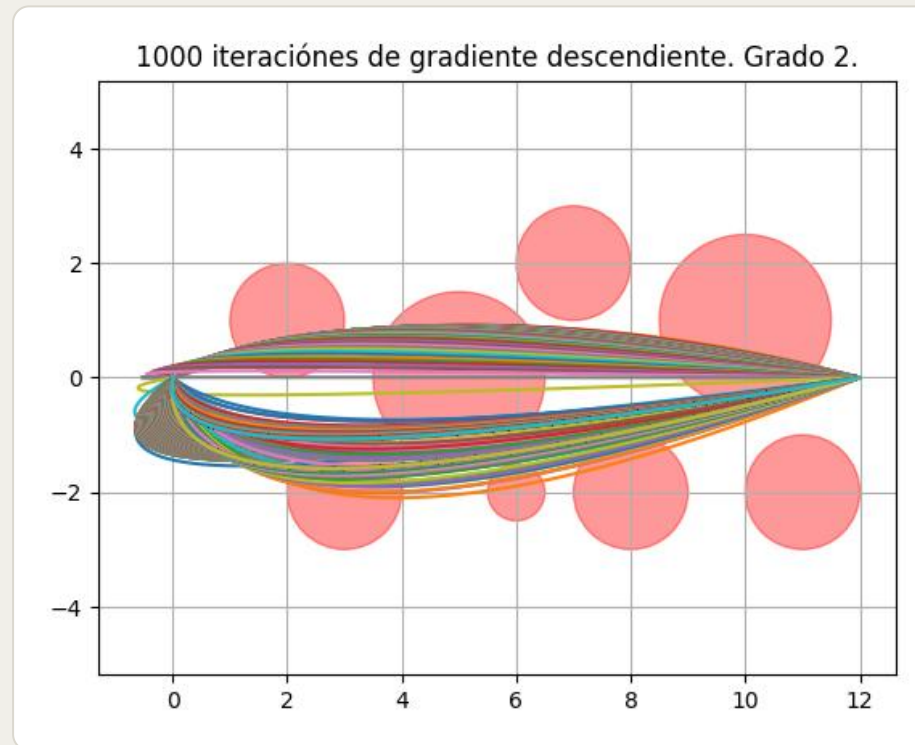
$$E_{\text{largo}}(\theta) = \sum_{i=1}^{N-1} \|B_n(t_{i+1}) - B_n(t_i)\|^2.$$

$$E_{\text{suavidad}}(\theta) = \sum_{k=1}^{n-1} \|P_{k-1} - 2P_k + P_{k+1}\|^2.$$

$$E_{\text{obstáculos}}(\theta) = \sum_{i=1}^N \sum_{j=1}^M [\text{máx}(0, r_j + \delta - \|B_n(t_i) - c_j\|)]^2.$$

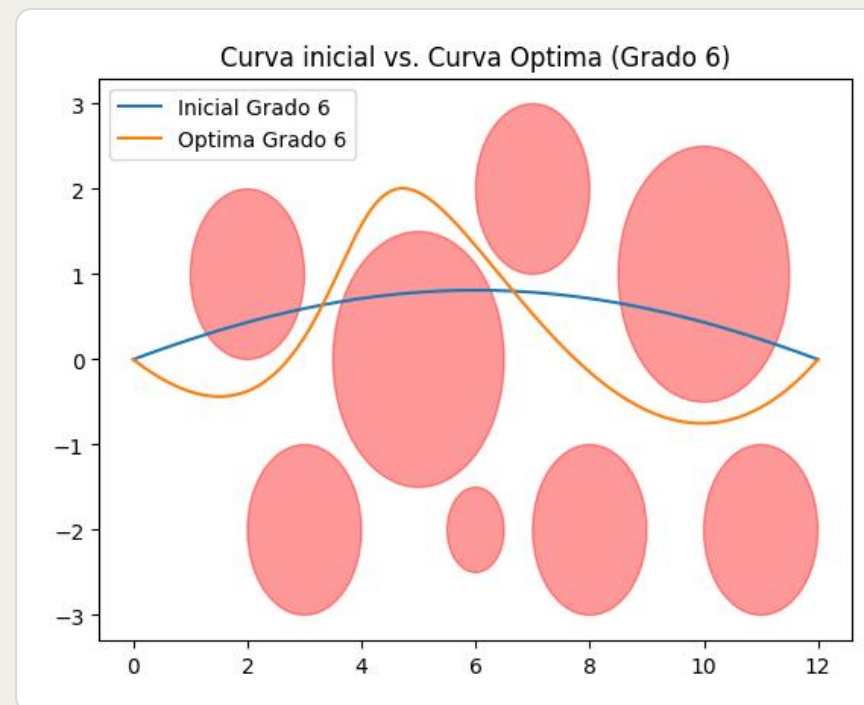
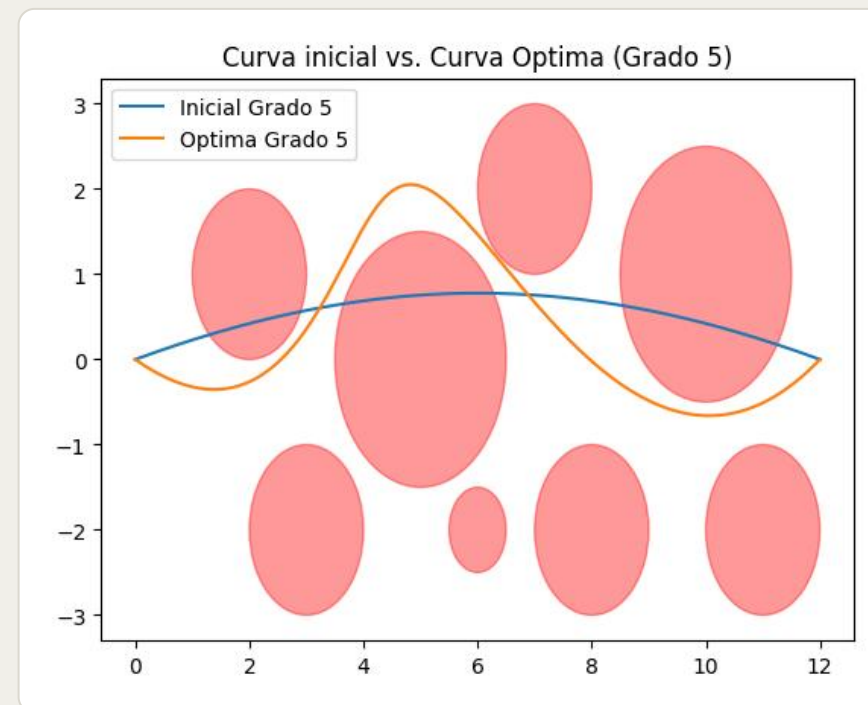
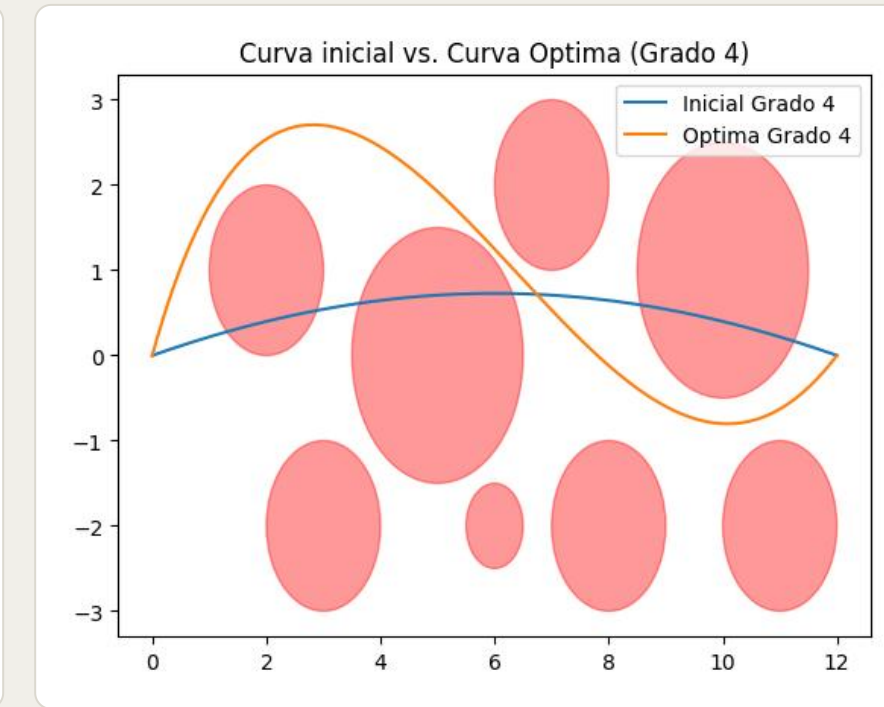
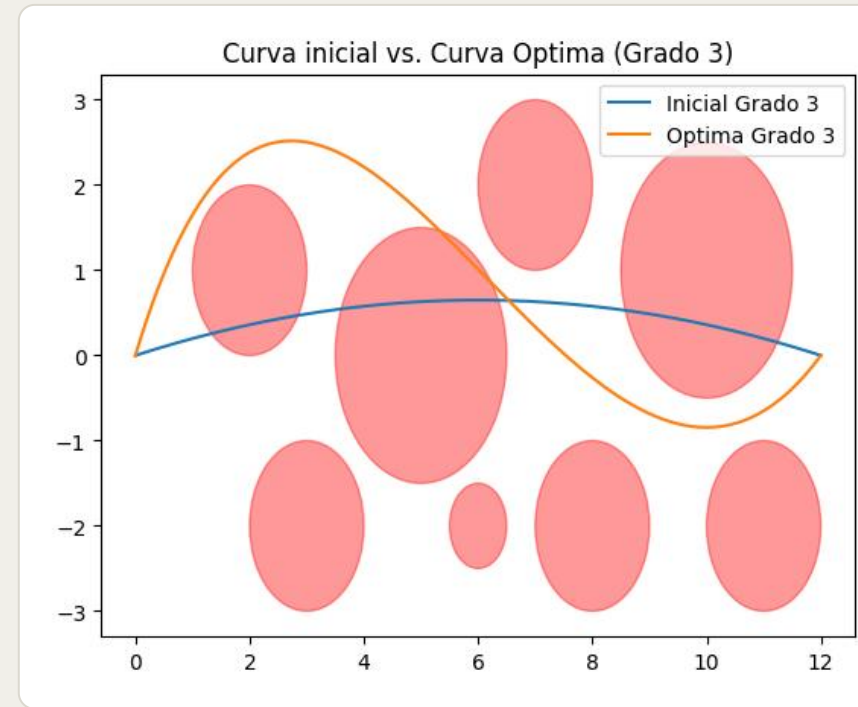
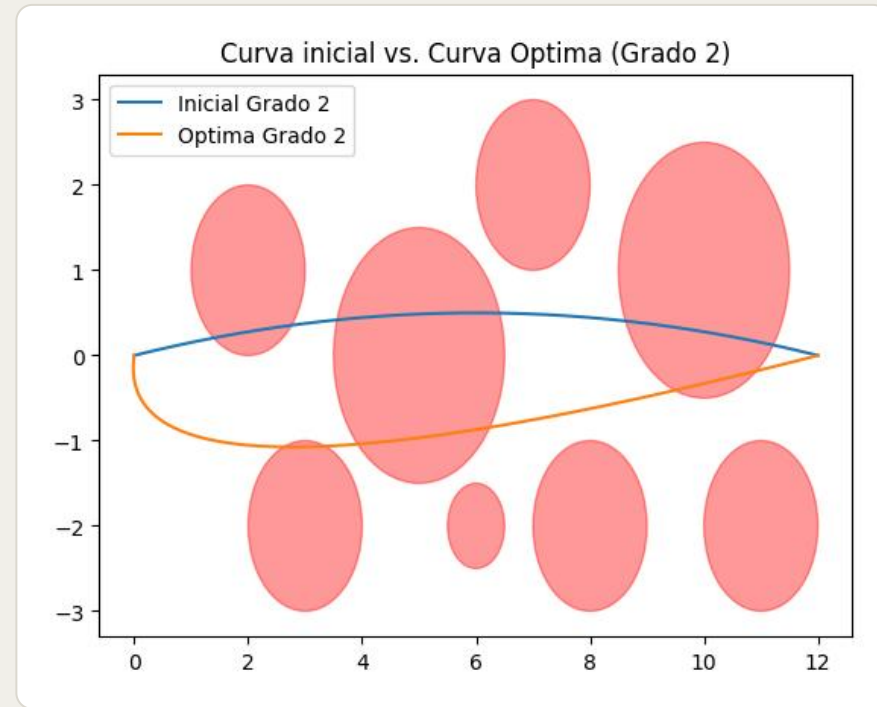
--> δ = variable para medir dist. a obs.

Optimización automática — 1000 iteraciones por grado



- $\lambda_{\text{suavidad}} = 0,1$, $\lambda_{\text{obs}} = 10$, $\delta = 0,3$, $a = 1$ (determina curva inicial), tasa = 0.01, iteraciones = 1000 y epsilon = 0.0001

Curva inicial vs. curva óptima por grado



- $\lambda_{\text{suavidad}} = 0,1$, $\lambda_{\text{obs}} = 10$, $\delta = 0,3$, $a = 1$ (determina curva inicial), $\text{tasa} = 0.01$, $\text{iteraciones} = 1000$ y $\text{epsilon} = 0.0001$

Trayectoria Factible y Menor grado para factibilidad

- Definimos como trayectoria factible:

--> No hay colisiones (lo chequeo en cada iteración con `colision()`)

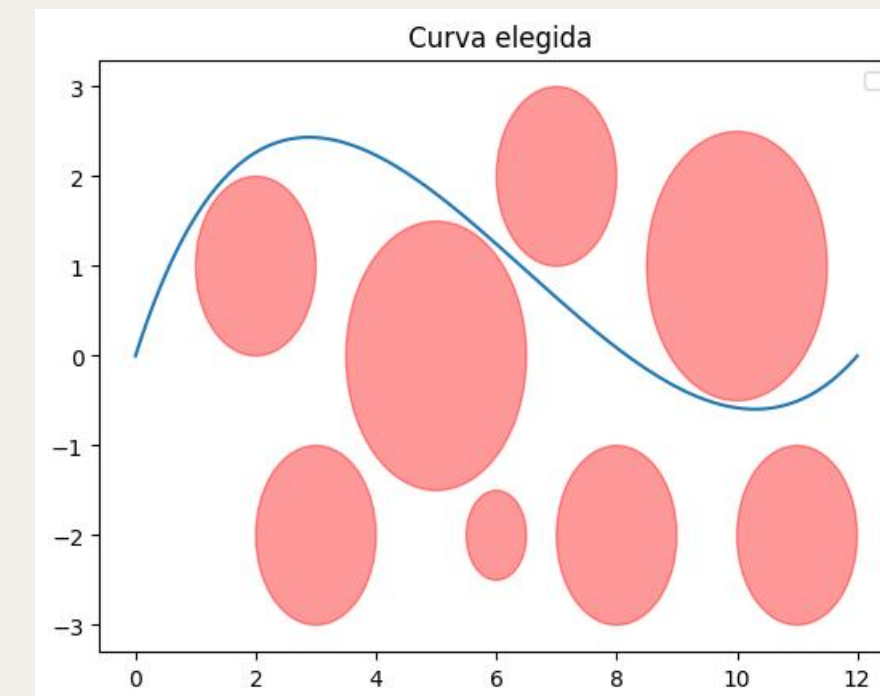
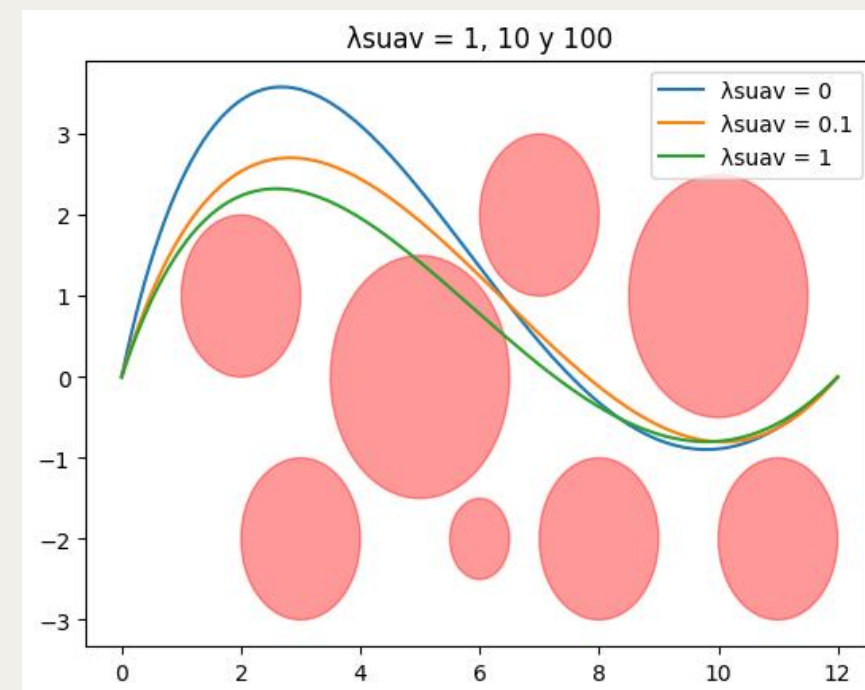
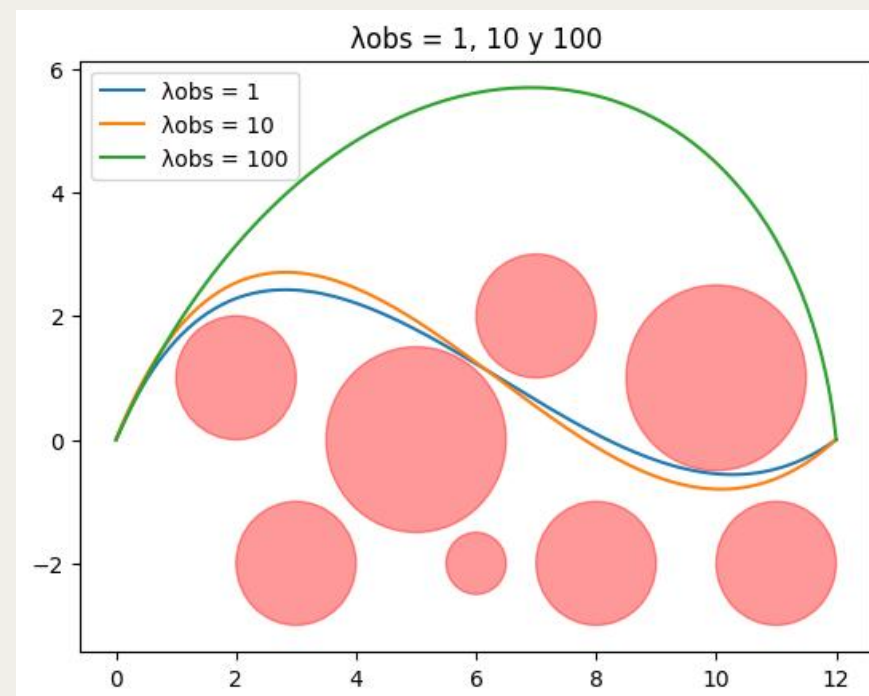
--> Distancia mínima a cualquier obstaculo sea al menos $\delta = 0.2$

--> Criterio de suavidad alto ($\lambda_{suav} = 0.1$) para que un robot realmente no tenga que hacer picos raros que puedan arruinar su trayectoria.

- El menor grado n para poder construir una trayectoria factible es grado 3. Con 1 solo punto distinto de los puntos de control (grado 2) no podemos encontrar una solución que cumpla el criterio de factibilidad.

Análisis de la trayectoria obtenida

- Cuando aumenta el grado, las curvas tienen mas picos, se da gracias a la cantidad de puntos de control.
 - Para λ_{obs} grandes se evita mucho la cercania a obstaculos, para λ_{obs} chicos se colisiona con obstaculos.
 - Para λ_{suav} grandes se evitan picos, para λ_{suav} hay picos.
- Curva elegida: 3 grados (simplicidad), $\lambda_{suav} = 0.1$ (no hayan picos), $\lambda_{obs} = 10$ (se evite cercania de obstaculos), tasa de aprendizaje = 0.01 (no diverge). Respeta nuestra trayectoria factible.



Conclusion del TP

En este trabajo recorrimos muchos temas que vimos en la materia pero especificamente en la segunda parte.

Primero, la idea de minimos cuadrados se ve claramente en la resolucion del problema usando metodo gradiente descendiente y metodo newton.

El error viene dado por la formula $\|Ap - q\|^2$, si igualamos la derivada a cero obtenemos las ecuaciones normales. En cada iteración del gradiente descendiente usamos las ecuaciones normales para acercarnos poco a poco al optimo.

En el metodo de newton resolvemos minimos cuadrados en una sola iteración. Calculando la formula de ecuaciones normales.

Para conectar con formas cuadraticas podemos ver que desarrollando la funcion del error obtenemos una forma cuadratica en $x^T A x$.

Cuando le calculamos los autovalores vemos que son todos positivos. Dando a entender que es positiva definida.

Si la matriz A es positiva definida significa que tenemos un unico minimo global.

Entonces, cuando hacemos metodo del gradiente y metodo de newton podemos factiblemente llegar a la mejor solución.

Optimización numerica: encontrar max o min de una función cuando no podes resolverlo a mano --> Metodo de newton y metodo de gradiente descendiente.

Regularización: En el ej 12. Regularizas el peso del error de la suavidad o del error obstaculo con λ_{obs} y λ_{suav} .